

Sports Person Image Classifier – Project Process Document

1. Project Objective

The objective of this project was to build an end-to-end machine learning application capable of identifying a sports personality from an uploaded image. The system was designed to detect a face in the image, extract useful features from the face, classify the face into one of the selected sports personalities, and present the result through a simple web interface.

The project was also intended as a hands-on portfolio project covering the full machine learning lifecycle:

- data preparation
 - image preprocessing
 - feature engineering
 - model training
 - model serialization
 - web application development
 - GitHub version control
 - cloud deployment
-

2. Project Inspiration

This project was inspired by a machine learning project walkthrough shared by the **Codebasics** YouTube channel. The project idea and learning flow were taken as inspiration, while the implementation, debugging, testing, GitHub setup, and Streamlit deployment were carried out independently as part of hands-on machine learning practice.

3. Problem Statement

The goal was to create a machine learning model that can identify a sports personality from an image by using classical computer vision and machine learning techniques.

The application needed to:

1. Accept an image from the user
2. Detect whether a valid face is present
3. Ensure at least two eyes are visible for reliable prediction

4. Extract image-based features
 5. Predict the sports person
 6. Display a confidence score
 7. Handle invalid cases such as side-face images or group photos gracefully
-

4. Sports Personalities Selected

The project was built as a multi-class image classifier for the following sports personalities:

- Lionel Messi
- Maria Sharapova
- Serena Williams
- Virat Kohli
- Roger Federer

These classes were mapped in a class dictionary file used during prediction.

5. Dataset Collection

The first phase of the project involved collecting images of the selected sports personalities. Images were gathered from publicly available sources and organized class-wise for model training.

Dataset collection goals:

- collect multiple images per sports person
- include clear frontal face images
- ensure enough variety in pose and expression
- avoid irrelevant or low-quality images as much as possible

Since raw internet images often contain noise, not all collected images were suitable for training directly. Therefore, a cleaning step was required.

6. Data Cleaning and Face Validation

To improve training quality, a face validation pipeline was created using **OpenCV Haar Cascade classifiers**.

Two cascades were used:

- haarcascade_frontalface_default.xml
- haarcascade_eye.xml

Cleaning rule:

Only keep images where:

- a face is detected
- at least **two eyes** are detected inside the face region

Why this step was important

Many raw images may contain:

- side faces
- group photos
- partial faces
- blurred images
- faces hidden by accessories or poor angles

Training on such noisy images reduces model quality. By keeping only images with a clear face and two visible eyes, the dataset quality improves and the feature extraction becomes more reliable.

7. Image Preprocessing Pipeline

The project used a utility function to preprocess the input image before classification.

Preprocessing steps:

1. Read image using OpenCV
2. Convert image to grayscale
3. Detect faces using Haar Cascade
4. For each detected face, detect eyes inside the face region
5. If two or more eyes are found, crop the face region
6. Return the cropped face image for feature extraction

If no face with at least two eyes was found, the application returned an error message instead of a prediction.

This behavior was later visible in testing when side-face and group images were rejected.

8. Feature Engineering

The model was built using a combination of:

1. **raw image features**
2. **wavelet transformed features**

This was done to capture both:

- basic visual appearance
 - high-frequency structural information from the face
-

9. Raw Image Features

The cropped face image was resized to:

- **32 × 32**

The resized face image contributed raw pixel features.

Since the image is in color, the feature vector from the raw image part was derived from:

- $32 \times 32 \times 3 = 3072$ values

These values capture the general visual representation of the face.

10. Wavelet Transform Features

In addition to raw pixel features, the project used a **wavelet transform** to extract structural patterns from the face.

The wavelet transformation was implemented using **PyWavelets**.

Parameters used:

- wavelet mode: db1
- level: 5

Wavelet process:

1. Convert image to grayscale
2. Normalize image values
3. Apply wavelet decomposition
4. Suppress approximation coefficients
5. Reconstruct the high-frequency image

6. Resize the transformed image to **32 × 32**

The wavelet-transformed image contributed:

- $32 \times 32 = 1024$ values

Why wavelet features were used

Wavelet transforms help highlight:

- edges
- facial structures
- texture patterns

This can improve the model's ability to distinguish between similar faces.

11. Final Feature Vector Construction

The final feature vector was created by vertically stacking:

- raw image features → 3072 values
- wavelet features → 1024 values

Total feature vector size:

- **4096 features**

This 4096-dimensional feature vector became the input to the machine learning model.

12. Model Training

The classification model used for this project was a **Support Vector Machine (SVM)**.

The training notebook handled:

- feature extraction for training images
- class label preparation
- model fitting
- model evaluation
- artifact generation

The trained model was saved using **Joblib**.

Saved artifacts:

- saved_model.pkl → trained SVM model
- class_dictionary.json → class label mapping

These artifacts were later loaded in the Streamlit application for inference.

13. Utility Module (`util.py`)

A separate utility module was created to keep the prediction logic organized.

The main responsibilities of `util.py` were:

13.1 `load_saved_artifacts()`

Loads:

- the trained model from `saved_model.pkl`
- the class dictionary from `class_dictionary.json`

13.2 `w2d()`

Applies wavelet transformation to an image.

13.3 `get_cropped_image_if_2_eyes()`

Reads an image, detects a face, checks for two eyes, and returns the cropped face image if valid.

13.4 `get_class_name()`

Maps the predicted class number back to the sports person name.

13.5 `classify_image()`

This is the main prediction function. It:

- validates the image
- extracts raw and wavelet features
- builds the final feature vector
- predicts the sports person
- calculates confidence using `predict_proba()`
- returns the result as a dictionary

Example output format:

```
{  
  "player": "Maria Sharapova",  
  "confidence": 78.72  
}
```

14. Streamlit Application Development

A web application was created using **Streamlit** to make the model accessible through a browser interface.

Features added in the app:

- project title and description
- image upload option
- uploaded image preview
- prediction result display
- confidence score display
- progress bar for confidence
- error handling for invalid images
- GitHub repository link

Typical app flow:

1. User uploads an image
 2. App temporarily stores the image
 3. Prediction pipeline is called
 4. If a valid face is found:
 - predicted player is shown
 - confidence score is shown
 5. If no valid face is found:
 - error message is displayed
-

15. Temporary File Handling

The uploaded image used temporarily for processing using Python's `tempfile` module.

The temporary image file was used only for prediction and then removed, ensuring that uploaded images were not permanently stored as part of the application flow.

This was an important privacy and cleanup consideration during app development.

16. Confidence Score Integration

The app was enhanced to show not just the predicted sports person but also a confidence score.

The confidence was calculated as:

- the maximum probability returned by `model.predict_proba()`
- converted to a percentage
- rounded to two decimal places

This confidence score was then displayed in the Streamlit UI along with a progress bar.

17. Edge Case Testing

After the first working version of the app was ready, multiple test cases were used to evaluate behavior.

Test cases included:

- frontal images of sports personalities
- personal images
- side-face images
- group photos

Observations:

- frontal sports images usually gave good predictions, often above 70% confidence
- personal images sometimes matched a player with lower confidence
- side-face images were often rejected because two eyes were not detected
- group photos were also rejected in many cases for the same reason

This testing helped confirm that the app handled invalid inputs appropriately.

18. Local Environment Issues Faced

During local development, several setup issues were encountered and resolved.

18.1 Python not recognized

Initially, `python --version` failed because the Windows PATH and App Execution Alias settings were interfering with the installed Python version.

Resolution:

- verified the actual Python installation path
- corrected PATH settings
- ensured the proper Python executable was being used

18.2 Missing packages

Errors such as the following were encountered:

- `ModuleNotFoundError: No module named 'joblib'`
- `ModuleNotFoundError: No module named 'cv2'`
- `ModuleNotFoundError: No module named 'pywt'`

Resolution:

Required dependencies were installed and verified using `pip show`.

19. Dependency Management

To support deployment and reproducibility, a `requirements.txt` file was created containing the project dependencies.

Final requirements used:

- `streamlit`
- `numpy`
- `opencv-python-headless`
- `PyWavelets`
- `scikit-learn`
- `joblib`

This file ensured that the Streamlit Cloud environment could install all required packages automatically.

20. Git and GitHub Integration

Version control was set up using Git.

Steps performed:

- initialized Git repository
- created `.gitignore`
- staged project files
- committed project files

- connected local repository to GitHub
- pushed code to remote repository

GitHub repository:

<https://github.com/ajinkyampawar/sports-person-image-classifier>

This allowed the project to be deployed from GitHub and made it available as a portfolio project.

21. Streamlit Cloud Deployment

The application was deployed using **Streamlit Community Cloud**.

Deployment flow:

1. push project to GitHub
 2. connect GitHub repository to Streamlit Cloud
 3. choose branch and app file
 4. allow Streamlit to install dependencies from requirements.txt
 5. run the app in the cloud
-

22. Deployment Issue Encountered

During deployment, the app initially failed with an OpenCV import error.

Error:

ImportError: libGL.so.1: cannot open shared object file

Cause:

The package opencv-python depends on certain GUI-related libraries that are not always available in cloud environments like Streamlit Community Cloud.

Resolution:

The dependency was changed from:

- opencv-python

to:

- opencv-python-headless

This version of OpenCV is more suitable for server/cloud environments because it excludes GUI-related dependencies.

After updating requirements.txt, committing the change, and pushing it to GitHub, the Streamlit deployment succeeded.

23. Final Application Behavior

Once deployed successfully, the app was able to:

- accept uploaded images
- detect valid frontal faces with two visible eyes
- generate predictions for known sports personalities
- show confidence scores
- reject unsupported edge cases such as side-face and group images

This completed the project as a working, cloud-hosted machine learning application.

24. Final Project Structure

The final project directory included:

- app.py
 - util.py
 - saved_model.pkl
 - class_dictionary.json
 - requirements.txt
 - .gitignore
 - Haar Cascade XML files
 - training notebook
 - README file
 - screenshots folder
-

25. Key Learnings from the Project

This project provided practical exposure to the following:

Machine Learning

- supervised image classification
- feature engineering using wavelets
- model training with SVM
- prediction probability interpretation

Computer Vision

- face detection using OpenCV Haar Cascades
- eye detection and validation
- image cropping and resizing

Python Development

- utility module design
- file handling
- temporary file cleanup
- dependency management

Deployment and Tooling

- Git and GitHub workflow
- creating requirements.txt
- debugging package issues
- Streamlit Community Cloud deployment
- cloud compatibility fixes for OpenCV

26. Future Improvements

Possible future enhancements for the project include:

1. Add **Top-3 predictions** instead of only the top prediction
2. Increase the size and quality of the training dataset
3. Support more sports personalities
4. Improve face detection using more robust modern detectors
5. Replace classical wavelet + SVM approach with deep learning embeddings or CNN-based classification
6. Improve app UI and result presentation
7. Add analytics or usage monitoring if the project is extended further

27. Conclusion

The Sports Person Image Classifier project successfully demonstrated the end-to-end development of a machine learning application, starting from image preprocessing and feature engineering all the way to deployment on the cloud.

The project combined:

- computer vision
- classical machine learning
- Streamlit app development
- GitHub version control
- cloud deployment

It also provided hands-on experience in debugging environment issues, resolving dependency problems, handling real-world edge cases, and turning a machine learning model into a usable product.

Overall, the project serves as a strong portfolio example of an end-to-end machine learning workflow implemented and deployed in practice.